Machine Learning Intelligent Chip Design

指導教授: 陳坤志

Final Project

NoC-based CNN architecture with AXI4 DMA architecture & optimized design

(結報)

系級 中央大學電機四 B

學號 B132501

姓名 翁瑄妤

一、 模擬結果

主要在工作站上執行程式，使用 TA 提供的輸入資料（在 data 檔裡）進行測試。此 project 成功執行一個支援 AXI4 DMA 架構的神經網路架構，並使用 Alexnet 架構處理影像分類，其結果如下：

- 模擬完整執行 9 層卷積與全連接層，經過 9 顆 PE 處理，逐層資料透過 NoC 進行傳遞。
- 所有層的輸入資料、權重與 bias 均透過 AXI4 Burst 傳輸進入各 PE 模組，並觀察到資料流穩定傳遞、模擬正確完成。

模擬中各層處理過程均輸出中介結果，並於最終輸出 softmax 結果，顯示前 100 個 imagenet_classes 機率。各層傳輸與處理均印出輸入大小、權重數量、輸出大小等訊息，輔助驗證每一層計算正確性。

Top 100 classes:

idx	val	possibility	class name
285	20.21	96.38	Egyptian cat
281	16.14	1.65	tabby
282	15.73	1.10	tiger cat
287	14.79	0.43	lynx
728	14.41	0.29	plastic bag
330	12.73	0.05	wood rabbit
331	12.19	0.03	hare
457	10.94	0.01	bow tie
335	10.67	0.01	fox squirrel
463	10.57	0.01	bucket
478	10.32	0.00	carton
876	10.29	0.00	tub
622	10.18	0.00	lens cap
904	10.01	0.00	window screen
700	9.56	0.00	paper towel
278	9.39	0.00	kit fox
8	9.29	0.00	hen
284	9.21	0.00	Siamese cat
722	8.80	0.00	ping-pong ball
434	8.77	0.00	bath towel
452	8.71	0.00	bonnet
289	8.31	0.00	snow leopard
753	8.30	0.00	radiator
681	8.11	0.00	notebook
673	8.04	0.00	mouse
840	8.03	0.00	swab
773	7.94	0.00	saltshaker
782	7.90	0.00	screen
896	7.71	0.00	washbasin
44	7.70	0.00	alligator lizard
719	7.69	0.00	piggy bank
138	7.68	0.00	bustard
998	7.57	0.00	ear
543	7.52	0.00	dumbbell
451	7.49	0.00	bolo tie
283	7.40	0.00	Persian cat
272	7.32	0.00	coyote
999	7.27	0.00	toilet tissue
769	7.23	0.00	rule
713	7.14	0.00	photocopier
760	7.12	0.00	refrigerator
474	7.08	0.00	cardigan
534	6.99	0.00	dishwasher
797	6.94	0.00	sleeping bag
552	6.93	0.00	feather boa
793	6.93	0.00	shower cap
322	6.92	0.00	ringlet
288	6.91	0.00	leopard
750	6.91	0.00	quilt
897	6.84	0.00	washer
36	6.83	0.00	terrapin
326	6.76	0.00	lycaenid
723	6.75	0.00	pinwheel
7	6.72	0.00	cock
333	6.66	0.00	hamster
519	6.65	0.00	crate
588	6.65	0.00	hamper
883	6.54	0.00	vase
725	6.50	0.00	pitcher
438	6.46	0.00	beaker
469	6.37	0.00	caldron
350	6.36	0.00	ibex
968	6.28	0.00	cup
280	6.25	0.00	grey fox
290	6.22	0.00	jaguar
83	6.18	0.00	prairie chicken
680	6.15	0.00	nipple
41	6.13	0.00	whiptail
572	6.13	0.00	goblet
742	6.07	0.00	printer
359	6.03	0.00	black-footed ferret
353	6.01	0.00	gazelle
937	5.96	0.00	broccoli
862	5.94	0.00	torch
286	5.91	0.00	cougar
504	5.86	0.00	coffee mug
855	5.81	0.00	thimble
332	5.74	0.00	Angora
400	5.73	0.00	academic gown
845	5.69	0.00	syringe
38	5.68	0.00	banded gecko
271	5.63	0.00	red wolf
357	5.61	0.00	mink
47	5.60	0.00	African chameleon
869	5.58	0.00	trench coat
584	5.57	0.00	hair slide
298	5.45	0.00	mongoose
761	5.39	0.00	remote control
470	5.38	0.00	candle
636	5.36	0.00	mailbag
620	5.35	0.00	laptop
268	5.31	0.00	Mexican hairless
273	5.31	0.00	dingo
514	5.28	0.00	cowboy boot
596	5.25	0.00	hatchet
885	5.24	0.00	velvet
929	5.23	0.00	ice lolly
899	5.22	0.00	water jug
473	5.20	0.00	can opener
377	5.15	0.00	marmoset

```
==========================
+                        +        |\__|\
+  Congratulations !!    +       / 0.0  |
+                        +     /_____   |
+  Simulation completed  +    /^ ^ ^ \  |
+  at 183994509 th cycle  +   |^ ^ ^ ^ |w|
+                        +    \m___m__|_|
==========================
```

二、AXI4 DMA 架構實作說明

設計總覽：

- Controller 作為 AXI Master，發起 AXI4 Read 請求
- ROM 作為 AXI Slave，回應讀取要求
- 傳輸資料再封裝為 flits，透過 NoC 傳遞給對應 PE

訊號定義與參數 (本設計只用到 Read Address Channel 使 Controller 能讀取 ROM 資料)：

- ARADDR：32-bit 起始記憶體位址，byte 為單位
- ARLEN：傳輸筆數，最大為 256 筆
- ARSIZE：單筆大小
- ARBURST：設為 01 表示 INCR 連續傳輸
- ARVALID / ARREADY：握手機制
- RDATA：ROM 回傳資料 (32-bit)
- RLAST：傳送最後一筆資料時拉高
- RVALID / RREADY：資料通道握手

Controller 控制流程：

- 根據目前 layer_id 與資料型別 (input, weight, bias)，決定傳輸總數量
- 根據 AXI 最大 burst 限制（256 筆），分批發送傳輸要求
- 收到資料後，存入 vector 再轉為 flit 傳送
- 傳送結束後進入下一筆資料流程

ROM 行為說明：

- 原始架構：ROM 啟動時預載所有資料並記憶體對應定址
- 優化架構：ROM 根據 layer_id/type 動態讀入檔案內容
- 接收到 ARVALID 時，啟動傳輸、產生 RDATA
- 根據 ARLEN 控制傳送筆數，並於最後一筆產生 RLAST

三、優化架構設計與效能分析

項目	原始架構	優化架構
ROM 設計	預載入所有參數，透過 map 定址	動態依據 layer_id/type 載入檔案內容，僅保留一組 buffer
傳輸流程	Controller 使用固定位址	Controller 使用 layer_id 控制搭配 count 分段傳輸
彈性擴展性	定址結構限制記憶體編排	支援 layer 擴展，只需對應檔案與邏輯編排
NoC 架構	3×3 NoC	4×4 NoC

ROM 設計：

原始架構

```
ROM: Loaded ./data/cat.txt at address 0x0
ROM: Loaded ./data/conv1_weight.txt at address 0x1000000
ROM: Loaded ./data/conv1_bias.txt at address 0x2000000
ROM: Loaded ./data/conv2_weight.txt at address 0x3000000
ROM: Loaded ./data/conv2_bias.txt at address 0x4000000
ROM: Loaded ./data/conv3_weight.txt at address 0x5000000
ROM: Loaded ./data/conv3_bias.txt at address 0x6000000
ROM: Loaded ./data/conv4_weight.txt at address 0x7000000
ROM: Loaded ./data/conv4_bias.txt at address 0x8000000
ROM: Loaded ./data/conv5_weight.txt at address 0x9000000
ROM: Loaded ./data/conv5_bias.txt at address 0xa000000
ROM: Loaded ./data/fc6_weight.txt at address 0xb000000
ROM: Loaded ./data/fc6_bias.txt at address 0x1b000000
ROM: Loaded ./data/fc7_weight.txt at address 0x2b000000
ROM: Loaded ./data/fc7_bias.txt at address 0x3b000000
ROM: Loaded ./data/fc8_weight.txt at address 0x4b000000
ROM: Loaded ./data/fc8_bias.txt at address 0x5b000000
ROM memory size: 61251368
```

優化架構

```
ROM: Loaded ./data/conv5_bias.txt at address 0x0
ROM memory size: 256
ROM: Loaded ./data/fc6_weight.txt at address 0x0
ROM memory size: 37748736
-----------------------------------
[Cycle 7870987] Core 5 compute start
input_data.size(): 43264
weight.size(): 589824
bias.size(): 256
core_id: 5 done!
result.size(): 9216
all data send to router!
[Cycle 7880208] Core 5 compute done, duration = 9221 cycles
-----------------------------------
ROM: Loaded ./data/fc6_bias.txt at address 0x0
ROM memory size: 4096
ROM: Loaded ./data/fc7_weight.txt at address 0x0
ROM memory size: 16777216
```

由此可看出預載入所有資料會導致記憶體變大，因此若能動態載入檔案內容，即可有效節省記憶體使用量，降低系統啟動時的負擔，並提升系統的彈性與可擴充性，特別適合記憶體受限的嵌入式或硬體平台。

傳輸流程：

原始架構

```
if(curr_layer < 9){
    // Determine memory address based on layer and type
    sc_lv<32> mem_addr;
    int expected_data_count = 0;

    if(curr_layer == 0) {
        mem_addr = INPUT_BASE_ADDR;
        expected_data_count = 150528; // 3*224*224
    }
    else if(curr_layer == 1) {
        if(data_type == WEIGHT) {
            mem_addr = CONV1_WEIGHT_ADDR;
            expected_data_count = 23232; // 64*3*11*11
        } else {
            mem_addr = CONV1_BIAS_ADDR;
            expected_data_count = 64;
        }
    }
    else if(curr_layer == 2) {
        if(data_type == WEIGHT) {
            mem_addr = CONV2_WEIGHT_ADDR;
            expected_data_count = 307200; // 192*64*5*5
```

優化架構

```
if(curr_layer < 9){
    layer_id.write(curr_layer);
    layer_id_type.write(curr_type);
    layer_id_valid.write(true);
    wait();
    layer_id_valid.write(false);
    // Determine memory address based on layer and type
    int expected_data_count = 0;

    if(curr_layer == 0) {
        expected_data_count = 150528; // 3*224*224
    }
    else if(curr_layer == 1) {
        if(data_type == WEIGHT) {
            expected_data_count = 23232; // 64*3*11*11
        } else {
            expected_data_count = 64;
        }
    }
```

原始架構中，Controller 透過事先定義好的 mem_addr，直接指定每筆輸入資料在 ROM 中的絕對位址，並依照層與資料類型（如 input、weight、bias）發出對應的 AXI 讀取指令；而在優化架構中，則改採 layer_id 搭配內部狀態機控制流程，根據每層資料需求量（count）分段進行 AXI burst 傳輸，不再依賴硬編碼位址，提升了彈性與可維護性。

彈性擴展性：

原始架構

```
// Load input data
load_file(DATA_PATH + IMAGE_FILE_NAME, INPUT_BASE_ADDR);

// Load conv layer weights and biases
for (int i = 1; i <= 5; i++) {
    stringstream ss;
    ss << i;
    uint32_t weight_addr = CONV1_WEIGHT_ADDR + (i-1) * 0x2000000;
    uint32_t bias_addr = CONV1_BIAS_ADDR + (i-1) * 0x2000000;

    load_file(DATA_PATH + "conv" + ss.str() + "_weight.txt", weight_addr
    load_file(DATA_PATH + "conv" + ss.str() + "_bias.txt", bias_addr);
}

// Load FC layer weights and biases
for (int i = 6; i <= 8; i++) {
    stringstream ss;
    ss << i;
    uint32_t weight_addr = FC6_WEIGHT_ADDR + (i-6) * 0x20000000;
    uint32_t bias_addr = FC6_BIAS_ADDR + (i-6) * 0x20000000;

    load_file(DATA_PATH + "fc" + ss.str() + "_weight.txt", weight_addr);
    load_file(DATA_PATH + "fc" + ss.str() + "_bias.txt", bias_addr);
```

優化架構

```
// on new layer_id_valid: Load corresponding file
if (layer_id_valid.read()) {
    // Read signals
    int id = layer_id.read();
    bool type = layer_id_type.read();
    string filename;
    if (id == 0)
    {
        filename = string(DATA_PATH) + IMAGE_FILE_NAME;
    }
    else if (id <= 5)
    {
        std::stringstream ss;
        ss << id;
        if (type == 0)
            filename = string(DATA_PATH) + "conv" + ss.str() + "_weight.txt";
        else
            filename = string(DATA_PATH) + "conv" + ss.str() + "_bias.txt";
    }
    else if (id <= 8)
    {
        std::stringstream ss;
        ss << id;
        if (type == 0)
            filename = string(DATA_PATH) + "fc" + ss.str() + "_weight.txt";
        else
            filename = string(DATA_PATH) + "fc" + ss.str() + "_bias.txt";
    }
    else
    {
        cout << "Error: Invalid layer id " << id << "." << endl;
        sc_stop();
    }
```

原始架構採用固定記憶體定址方式，將每層資料對應至事先規劃好的記憶體位址，雖可快速存取，但在新增層數或調整模型架構時，必須重新修改記憶體映射與程式碼；相較之下，優化架構採用以 layer_id 搭配檔名推導的機制，自動載入對應檔案並依需求決定傳輸資料量，僅需新增資料檔與調整邏輯條件即可支援擴展，大幅提升系統彈性與可維護性。

NoC 架構：

原始架構

```
============================
+                          +        |\__|\
+   Congratulations !!     +       / 0.0  |
+                          +     /_____   |
+   Simulation completed   +    /^ ^ ^ \  |
+   at 183994509 th cycle   +   |^ ^ ^ ^ |w|
+                          +    \m___m__|_|
============================
```

優化架構

```
============================
+                          +        |\__|\
+   Congratulations !!     +       / 0.0  |
+                          +     /_____   |
+   Simulation completed   +    /^ ^ ^ \  |
+   at 183994497 th cycle   +   |^ ^ ^ ^ |w|
+                          +    \m___m__|_|
============================
```

原始架構採用 3×3 NoC 設計，剛好對應 9 層模型運算，每個 PE 處理一層，路由結構簡單、連接清楚，有效避免資源浪費，最終模擬完成於 cycle 183,994,509；而優化架構為 4×4 NoC，雖增加了額外的 Router 與連線數，但保留了更高的彈性以支援多層擴展或多資料來源處理，其最終模擬完成於 cycle 183,994,497，效能近乎相同，差異主要體現在擴展性與資源使用上。

AXI 傳輸行為對比：

- 兩者均支援 burst 傳輸
- 原始架構傳輸位置精確、定址明確
- 優化後為動態 layer 管理，Controller 對傳輸位置完全主導

模擬分析與效能評估：

- 總執行時間相當，優化後架構在靈活性與維護上明顯提升

- 優化 ROM 結構後，程式碼精簡且利於除錯
- 模擬穩定性維持，分類準確率一致

四、遇到的挑戰與解決方案

1. AXI4 傳輸協定複雜、容易出錯

一開始最大的挑戰來自 AXI4 的握手機制。之前雖然學過 AXI，但真的實作時才發現光是 Read Transaction 就有很多細節，像是 ARVALID 和 ARREADY 的時機要對得剛剛好，傳輸時 ARLEN 要扣一，RLAST 必須在最後一筆送出，稍有錯誤就會造成 Controller 收不到資料。實作 burst 傳輸時更難，因為要根據 burst_len 計算地址偏移，還得同步資料回傳 timing。這部分我 debug 很久，後來靠在每次傳輸時印出當前位址、長度，並觀察 RVALID/RREADY 是否搭配成功，才慢慢讓資料順利回傳。

2. ROM 記憶體架構

原始設計的 ROM 是用 std::map 把所有資料一次載入進來，但儲存數量會很大，且若是使用固定位址且當層預留空間不夠時，會覆蓋掉下一層的資料。我一開始還以為是 flit 傳輸錯了導致在 Conv 5 之前的結果是正確的，然而 FC 6 之後卻有一些地方有錯誤，結果 trace 了好幾天才發現是 FC 6 預留空間不夠。最後決定改成「只載入目前要傳輸的 layer」，用 file_buf 暫存，用完就清掉，不但省記憶體，也不用擔心預留空間不足的問題。

五、心得

這次 Final Project 雖然難度非常高，debug 也花了比想像中還多的時間，但整體來說卻是我在這堂課所有作業中學到最多、收穫最扎實的一次。前面的幾次作業已經做過 CNN 架構，包括卷積層、全連接層、甚至 mesh-based NoC 的設計，Controller 與 ROM 的資料搬移流程也練習過不少。但這次最大的挑戰是要把原本簡化的資料傳輸架構，全面改寫為支援 AXI4-DMA 的方式，並整合到整個 CNN 流程中。這等於不是單純 patch 原本的 code，而是要從底層協定、資料分段機制，到高層資料驅動流程整個重構。

我覺得這次作業最有價值的地方在於它讓我真的理解到什麼叫「資料驅動的系統架構」。以前在寫 CNN 程式時，資料都存在變數裡，只要丟進函數就會有結果。但在這裡，光是讓一筆 float 資料從 ROM 經過 AXI4 傳輸，轉成 flit，再穿過 NoC，到達正確的 PE，被正確解包、計算、送回 Controller，就得經過非常多層的邏輯與通訊協定。這讓我開始理解，資料如何「走」在整個系統裡，其實才是真正的系統設計核心。

整個實作過程中，我也更熟悉 AXI4 的實作方式與時序邏輯，像是 burst 傳輸的封包管理、RLAST 處理、Ready/Valid 的配對等，都是過去只有短暫學習過但沒深入實作的內容。NoC 中的 flit-based 通訊更是讓我學到很多關於 Router 選擇、flow control、buffer 管理等細節。尤其當整個 pipeline 跑完，Controller 收到並印出正確的 softmax 的分類結果，那一刻真的有一種「全系統都串起來了」的爽感。

我很喜歡這種從演算法到系統架構的完整實作過程，不再只是寫幾行 model 定義就結束，而是從最底層的資料格式、傳輸機制，到最上層的推論邏輯都要自己動手架起來。這次

Final Project 不僅強化了我對 SystemC 的掌握，也讓我在 AXI4 協定與模組通訊設計上有了實戰經驗。雖然過程中真的踩了很多雷、也 debug 到懷疑人生，但能看到系統順利跑完、資料正確流動，那種成就感是過去從沒體驗過的。這次的 Final Project 絕對是我最辛苦但也最值得的一次實作經驗。